

Guia Técnico Saibai — Para Aprendizado

Projeto: Empório Saibai — Tema Shopify Oficial

Versão do tema: 1.2.4

Loja: emporiosaibai.com.br

Elaborado por: Veltrus Growth & Technology

Público: Everton (aprendiz de desenvolvimento de software)

Atualizado: junho 2026

Como usar este guia

Este documento foi escrito para você **aprender o projeto de verdade**, não apenas copiar comandos. Leia na ordem das seções na primeira passagem. Depois, use como referência quando for editar algo.

Regra de ouro: **se não entende o que um arquivo faz, não publique na loja live.**

1. Visão técnica executiva

O que é este projeto

O Empório Saibai é uma loja virtual na plataforma **Shopify**. O código deste repositório é o **tema** — ou seja, tudo que o visitante vê e interage no site: header, home, páginas de produto, carrinho, footer, banner de cookies, estilos e scripts.

A Veltrus personalizou um tema comercial chamado **Ella** (fornecido pela Halothemes) e criou uma camada própria identificada pelo prefixo **saibai-***. O resultado é um e-commerce pronto para vender alcachofras frescas e conservas, com identidade visual Saibai, páginas legais, tracking preparado e governança técnica.

O que significa desenvolvimento de tema Shopify

Desenvolver um tema Shopify não é “só mudar cores no painel”. Envolve:

- **Liquid** — linguagem de template que mistura HTML com dados da loja (produto, preço, menu, etc.)
- **CSS** — aparência visual
- **JavaScript** — interações (carrinho drawer, popup de cupom, consentimento LGPD)
- **JSON** — configuração de quais seções aparecem em cada página
- **Configurações do Admin** — pagamentos, frete, impostos, domínio (fora deste repositório)

De que um tema Shopify é feito

Um tema típico tem estas pastas principais (detalhadas na seção 2):

Pasta	Função resumida
layout/	Esqueleto HTML de todas as páginas
templates/	Qual página usa quais seções
sections/	Blocos grandes reutilizáveis (header, hero, footer)
snippets/	Pequenos pedaços reutilizáveis (logo, ícones de pagamento)
assets/	CSS, JS, imagens
config/	Configurações editáveis no Theme Editor
locales/	Traduções

Por que isto não é “só personalização visual”

No Saibai, além da identidade visual, a Veltrus entregou:

- Arquitetura modular com prefixo `saibai-*` (centenas de arquivos organizados)
- Performance (CSS crítico, preload de imagens, skip de `base.css` em templates Saibai)
- LGPD com Consent Mode v2 e banner próprio
- Anti-duplicação de tracking (IDs GA4/Meta vazios no tema; eventos via canais Shopify)
- App Proxy server-side (`/apps/vlt-tracking/events`)
- Páginas institucionais e links legais no footer
- Remoção de Boleto do storefront (método inativo no checkout)
- Theme Check com **0 offenses**
- Fluxo Git com PRs controladas

Personalizar só logo e banner não produz esse nível de controle de risco.

2. Mapa do repositório

layout/

O que faz: Define o HTML base que envolve todas as páginas (`<html>` , `<head>` , `<body>`). É o “esqueleto” fixo.

Quando editar: Ao adicionar scripts globais, meta tags, ou alterar a ordem de carregamento de CSS/JS.

O que pode quebrar: Scripts no `<head>` sem `defer` podem travar a página; remover `{{ content_for_header }}` quebra apps Shopify; alterar Consent Mode default pode violar LGPD.

Exemplo Saibai: `layout/theme.liquid` contém:

- Preload do hero WebP na home
- Consent Mode v2 default(`analytics_storage: 'denied'`)
- Grupos de seções: `header-group` , `popup-group` , `footer-group`
- Banner LGPD: `{% section 'saibai-consent-popup' %}`

templates/

O que faz: Diz **qual combinação de seções** compõe cada tipo de página (home, produto, carrinho, página institucional).

Quando editar: Ao reorganizar a home, criar nova página com template dedicado, ou adicionar/remover seções.

O que pode quebrar: JSON inválido impede o tema de publicar; remover uma seção referenciada gera erro no Theme Editor.

Exemplo Saibai:

- `templates/index.json` — home com `saibai-home-hero` , `saibai-home-editorial` , `saibai-home-products` , etc.
- `templates/page.politica-de-entrega.json` — página de política de entrega
- `templates/product.json` — página de produto (PDP)
- `templates/cart.json` — página do carrinho

sections/

O que faz: Blocos grandes com HTML, Liquid, schema (configurações no admin) e às vezes CSS/JS próprios. Podem ser arrastados no Theme Editor.

Quando editar: Ao mudar estrutura de um módulo inteiro (footer, hero, header).

O que pode quebrar: Schema inválido; `enabled_on` errado faz a seção sumir; lógica Liquid incorreta quebra renderização.

Exemplo Saibai:

- `sections/saibai-header.liquid` — header completo
- `sections/saibai-footer.liquid` — footer com newsletter e links legais

- `sections/saibai-home-hero.liquid` — hero da home
- `sections/saibai-consent-popup.liquid` — banner LGPD

snippets/

O que faz: Pedacos pequenos incluídos via `{% render 'nome-do-snippet' %}`. Não aparecem sozinhos no Theme Editor.

Quando editar: Para ajustes pontuais reutilizados em vários lugares (logo, CSS loaders, ícones de pagamento).

O que pode quebrar: Snippet inexistente gera erro fatal; parâmetros errados no `render` causam variáveis vazias.

Exemplo Saibai:

- `snippets/saibai-logo.liquid` — logo PNG via CDN
- `snippets/saibai-shell-css.liquid` — carrega CSS modular da camada Saibai
- `snippets/saibai-payment-icons.liquid` — selos Pix, Visa, MC, Amex, Elo
- `snippets/saibai-tracking-config.liquid` — JSON de config para tracking

blocks/

O que faz: No Shopify 2.0, blocos são sub-componentes **dentro** de seções (ex.: cada slide do hero, cada card editorial).

Quando editar: Ao mudar um item repetível dentro de uma seção configurável no admin.

O que pode quebrar: Tipos de bloco não declarados no schema da seção pai.

Exemplo Saibai: Blocos em `templates/index.json` dentro de `saibai-home-editorial` (`collection` , `lifestyle` , etc.).

assets/

O que faz: Arquivos estáticos — CSS, JS, imagens, SVGs, WebP.

Quando editar: Estilos visuais, comportamentos JS, trocar imagens do tema.

O que pode quebrar: Caminho errado no Liquid (`asset_url`); JS com erro para toda a página; CSS com especificidade errada “some” com o visual.

Exemplo Saibai:

- `assets/saibai-tokens.css` — design tokens (cores, espaçamentos)
- `assets/saibai-header.js` — comportamento do header
- `assets/saibai-consent-popup.js` — LGPD + carregamento condicional GA4/Meta
- `assets/saibai-hero-fresca-*.webp` — imagens otimizadas do hero

config/

O que faz: `settings_schema.json` define campos no Theme Editor; `settings_data.json` guarda valores salvos.

Quando editar: Para adicionar novas opções configuráveis pelo merchant (raramente pelo aprendiz sem revisão).

O que pode quebrar: JSON inválido; IDs duplicados; campos de tracking preenchidos por engano (duplica conversões).

Exemplo Saibai: Grupo “Saibai Tracking” com aviso explícito para manter `meta_pixel_id` e `ga4_measurement_id` **vazios** na operação normal.

locales/

O que faz: Traduções de textos do tema (`pt-BR.json` , `en.default.json`).

Quando editar: Para corrigir textos do checkout, labels de formulário, mensagens de erro.

O que pode quebrar: Chave errada não traduz; JSON grande com vírgula faltando quebra o tema.

Exemplo Saibai: `locales/pt-BR.json` com mais de 2.500 strings traduzidas para checkout e sistema.

`docs/`

O que faz: Documentação do projeto (guias, relatórios, smoke tests, backlog). **Não afeta a loja** — só informa a equipe.

Quando editar: Sempre que registrar entregas, auditorias ou instruções.

Exemplo Saibai: Este guia, `SMOKE_TEST_v1.2.4.md`, `DESPACHO_SAIBAI.md`, `BACKLOG_SAIBAI.md`.

`ops/` (externo ao repo do tema)

O que faz: Scripts operacionais Python para configurar a loja via API (legal, checkout, pagamentos, notificações). Fica em `../ops/scripts/` conforme README.

Quando editar: Apenas com aprovação — altera configurações reais da loja.

O que pode quebrar: Pagamentos, políticas legais, privacidade, checkout.

`tracking/` (externo / infraestrutura Veltrus)

O que faz: Endpoint server-side para eventos de marketing (GA4 Measurement Protocol, Meta CAPI). No storefront, o navegador envia eventos para `/apps/vlt-tracking/events` via **App Proxy** Shopify.

Quando editar: Não no tema — é backend/infra. No tema, só validar que o proxy responde **202 Accepted**.

3. Arquitetura de tema Shopify explicada

Liquid

Liquid é a linguagem de template da Shopify. Mistura HTML com tags especiais:

```
<h1>{{ product.title }}</h1>
<p>{{ product.price | money }}</p>
{% if product.available %}
  <button>Comprar</button>
{% endif %}
```

- `{{ }}` — imprime valor
- `{% %}` — lógica (if, for, assign)
- `| money` — filtros transformam dados

JSON templates

Arquivos como `templates/index.json` listam seções e suas configurações. O Shopify monta a página juntando layout + template + seções.

Sections vs snippets

	Section	Snippet
Onde aparece	Theme Editor, grupos (header/footer)	Só via código
Schema próprio	Sim	Não (recebe parâmetros)
Uso típico	Header, footer, hero	Logo, um trecho de CSS

Assets

CSS e JS ficam em `assets/` e são referenciados assim:

```
{{ 'saibai-hero.css' | asset_url | stylesheet_tag }}
<script src="{{ 'saibai-header.js' | asset_url }}" defer="defer"></script>
```

Schema

Bloco JSON no final de uma section que define campos editáveis no admin (textos, imagens, menus). Exemplo em `sections/saibai-header.liquid` — largura do logo, menu principal.

Theme settings

Valores globais do tema (`settings.logo_width` , `settings.meta_pixel_id`). Definidos em `config/settings_schema.json` , valores em `settings_data.json` .

App embeds

Apps Shopify podem injetar código via `{{ content_for_header }}` e blocos de app. Não remover `content_for_header` — quebra checkout, pixels de apps, etc.

Customer Events

Canal oficial Shopify para eventos no checkout (Purchase, AddToCart no checkout). O Saibai usa pixel customizado documentado em `ops/scripts/` — **não duplicar** com IDs no tema.

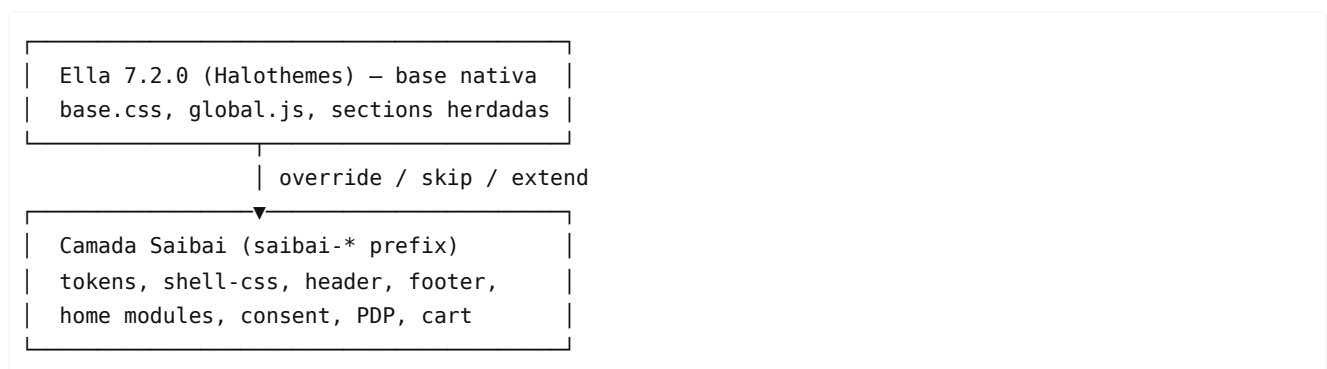
Admin vs arquivos de código

No código (tema)	No Admin Shopify
Visual, textos de seções, LGPD banner	Pagamentos, frete, impostos
Estrutura de páginas	Produtos, estoque
CSS/JS storefront	Domínio, SSL
Links do footer (menus)	Políticas legais oficiais (/policies/*)

Muitas coisas **só existem no Admin** e não aparecem neste Git.

4. Arquitetura Saibai

Base Ella + camada Saibai



Estratégia do prefixo `saibai-*`

Todo arquivo customizado da Veltrus usa o prefixo `saibai-` (ou comentário “Empório Saibai — Tema Oficial”). Isso permite:

- Saber o que é custom vs nativo Ella

- Buscar no projeto: `grep -r "saibai-" assets/`
- Evitar editar arquivos nativos sem necessidade

Por que camada em vez de reescrever tudo

- **Menor risco:** Ela já traz carrinho, busca, variantes, compare, quick-view
- **Atualizações:** Possível portar fixes do fornecedor em arquivos nativos
- **Performance:** `global-css.liquid` **pula** `base.css` em templates Saibai (home, collection, cart...) ganhando velocidade
- **Custo:** Reescrever um tema Shopify do zero levaria meses

Anti-duplicação GA4 / Meta no tema

Os campos **Saibai Tracking** no Theme Editor devem ficar **vazios** porque:

1. Google Analytics e Meta Pixel já podem estar ativos em **Vendas online → Preferências** (canais Shopify)
2. Preencher os campos dispara tags **extras** pelo banner LGPD (`saibai-consent-popup.js`)
3. Purchase no checkout deve ir pelo **Customer Events pixel**, não pelo tema

O snippet `saibai-tracking-config.liquid` expõe os IDs para o JS de consentimento — se vazios, GA4/Meta do tema **não** carregam.

Tracking server-side (visão geral)

```
Visitante → evento no navegador (com consentimento)
           → POST /apps/vlt-tracking/events (App Proxy)
           → Backend Veltrus (sGTM / worker)
           → GA4 Measurement Protocol + Meta CAPI
```

App Proxy: Rota `/apps/...` na loja que Shopify encaminha para servidor externo. O visitante vê URL do domínio Saibai; o processamento é server-side.

Por que 202 é importante: HTTP 202 = “aceito para processamento”. Confirma que o proxy está vivo e recebendo eventos sem erro 404/500.

Consentimento / LGPD

1. `layout/theme.liquid` define Consent Mode v2 **negado por padrão**
2. Banner `saibai-consent-popup` pergunta analytics/marketing
3. `saibai-consent-popup.js` registra consentimento via **Shopify Customer Privacy API**
4. Só então carrega GA4/Meta **se** IDs estiverem preenchidos (normalmente não estão)
5. Preferências salvas em `localStorage` (`saibai_consent_v1`)

Sem consentimento prévio, tracking de marketing é bloqueado — requisito LGPD.

5. Módulos visuais principais

Header

Item	Detalhe
Arquivos	<code>sections/saibai-header.liquid</code> , <code>snippets/saibai-header.liquid</code> , <code>assets/saibai-header*.css</code> , <code>assets/saibai-header.js</code>
Função	Logo, navegação, mega menu, busca, conta, sacola
Risco	Alto — afeta todas as páginas

Edição segura	Textos de menu via Admin → Navegação; largura logo via schema da section
Não tocar	Lógica de drawer mobile, lazy search na home sem entender saibai-search-lazy.js

Home / KV (key visual)

Item	Detalhe
Arquivos	templates/index.json, sections/saibai-home-*, assets/saibai-hero.css, saibai-hero.js
Função	Hero vídeo/imagem, editoriais, vitrine produtos, farm, feature, conservas
Risco	Médio-alto — primeira impressão, LCP
Edição segura	Textos/blocos no Theme Editor; imagens WebP em assets/
Não tocar	Ordem de preload em theme.liquid; remoção de seções sem testar mobile

Cards de produto

Item	Detalhe
Arquivos	snippets/saibai-prod-card.liquid, assets/saibai-shell-cards-*.css, component-card.css
Função	Imagem 1:1, preço, badge, botão comprar
Risco	Médio — vitrines home, PLP, relacionados
Edição segura	Ajustes de espaçamento com DevTools primeiro
Não tocar	aspect-ratio e object-fit sem QA — cards deformam

PDP (página de produto)

Item	Detalhe
Arquivos	templates/product.json, sections/main-product.liquid, sections/saibai-pdp-details.liquid, assets/saibai-pages-pdp-*.css, saibai-pdp-shipping.js
Função	Galeria, variantes, frete BR, abas, sticky ATC mobile
Risco	Alto — conversão direta
Edição segura	Textos de abas via metafields/snippets de política
Não tocar	Formulário add-to-cart; tracking gated por meta_pixel_id

Cart drawer

Item	Detalhe
Arquivos	assets/cart-drawer.js, assets/component-cart-drawer.css, snippets Ella herdados
Função	Carrinho lateral ao adicionar produto
Risco	Alto
Edição segura	Textos de confiança, hint de cupom (saibai-cart-coupon-hint)

Não tocar	Event listeners de qty/remove; integração com cart.js
-----------	---

Página do carrinho

Item	Detalhe
Arquivos	templates/cart.json, sections de cart Ella + CSS Saibai em saibai-pages-css
Função	Revisão de itens, cupom, checkout
Risco	Alto
Edição segura	Copy de frete grátis, mensagens Saibai
Não tocar	Botão checkout e formulários de qty

Footer

Item	Detalhe
Arquivos	sections/saibai-footer.liquid, assets/saibai-footer-*.css, snippets/saibai-payment-icons.liquid
Função	Newsletter, colunas de links, selos pagamento, crédito Veltrus, preferências cookies
Risco	Médio
Edição segura	Menus via Admin; textos newsletter no schema
Não tocar	Fallback de payment icons sem validar métodos ativos no checkout

Páginas legais / institucionais

Item	Detalhe
Arquivos	templates/page.politica-*.json, sections/saibai-policy-*.liquid, snippets/saibai-policy-*.liquid
Função	Entrega, troca, privacidade, sobre, contato
Risco	Alto (jurídico)
Edição segura	Nenhuma sem aprovação do dono
Não tocar	Criar páginas duplicadas de políticas que já existem em /policies/*

Banner cookie / LGPD

Item	Detalhe
Arquivos	sections/saibai-consent-popup.liquid, assets/saibai-consent-popup.js, CSS saibai-consent-popup-*.css
Função	Consentimento analytics/marketing, integração Shopify Privacy API
Risco	Crítico (legal + tracking)
Edição segura	Textos do banner com revisão jurídica
Não tocar	Fluxo setTrackingConsent, ordem de carregamento de pixels

Checkout branding

Item	Detalhe
Arquivos	Admin → Configurações → Checkout (não é arquivo do tema no Basic Shopify)
Função	Cores Saibai (#76BD22), logo, fonte Jost no checkout e contas cliente
Risco	Alto
Edição segura	Via admin com guia docs/CHECKOUT_BRANDING_GUIA.md
Não tocar	Métodos de pagamento, ordem de gateways

6. CSS e cascata

O que é cascata

O navegador decide qual regra CSS “vence” quando várias se aplicam ao mesmo elemento. Ordem importa: **última regra compatível com maior especificidade ganha**.

Por que CSS antigo do Ella sobrescreve código novo

O Ella carrega `base.css` (milhares de linhas) com seletores amplos. Mesmo com arquivo Saibai depois, seletores mais específicos do Ella podem vencer.

O que é `!important`

Força uma propriedade a ter prioridade máxima naquela declaração. O Saibai usa **~897 ocorrências** em CSS custom — é **intencional** para vencer o tema nativo.

Por que refactor exige QA visual

Remover `!important` ou consolidar arquivos sem testar pode:

- Botões voltarem à cor verde padrão Shopify
- Cards perderem proporção 1:1
- Header quebrar no mobile 375px

Design tokens

Variáveis centralizadas de cor, espaçamento, transição:

- `assets/saibai-tokens.css` — `:root` e tokens globais
- `assets/saibai-shell-kv.css` — reassert em `body.saibai-theme`

Tokens evitam `#76BD22` espalhado em 50 arquivos — mas no Saibai ainda há duplicação proposital entre arquivos para vencer `color-scheme` do Ella.

Evitar “correção sobre correção”

Mau padrão:

```
.btn { color: red !important; }
/* não funcionou */
html body .btn { color: blue !important; }
/* ainda não */
html body div .btn { color: green !important; }
```

Bom padrão:

1. Inspeccionar no DevTools qual regra está vencendo

2. Entender o seletor vencedor
3. Ajustar no arquivo Saibai correto (`saibai-shell-*` ou `saibai-pages-*`)
4. Preferir token + seletor com especificidade mínima necessária

Consolidar causas raiz

Antes de adicionar mais CSS, pergunte:

- O problema é ordem de carregamento? (`saibai-shell-css.liquid`)
- É template errado carregando `base.css` ?
- É setting do Theme Editor conflitando?

7. JavaScript

O que o JS do tema faz

- Carrinho (add, update, drawer)
- Galeria PDP, variantes
- Busca preditiva
- Popups (cupom, consentimento)
- Anúncios, hero vídeo
- Lazy load de scripts na home

Padrão do projeto

- IIFE: `(function () { 'use strict'; ... })()`;
- Sem arrow functions em código Saibai crítico (compatibilidade)
- `var` em vez de `let/const` nos arquivos Saibai legacy
- `defer` nos scripts no HTML

Riscos

Risco	Consequência
Listener duplicado	Add-to-cart dispara 2x; tracking duplicado
Script sem defer no <head>	Bloqueia renderização
Editar <code>cart-drawer.js</code> sem testar	Carrinho para de abrir

Debug no console do navegador

1. F12 → aba **Console** — erros em vermelho
2. Aba **Network** — CSS/JS 404, POST para `/apps/vlt-tracking/events`
3. `localStorage.getItem('saibai_consent_v1')` — ver consentimento salvo
4. `window.dataLayer` — eventos GA4
5. Erro conhecido em backlog: `ModalDialog is not defined` no quick-view (pré-existente)

8. Tracking

Client-side vs server-side

	Client-side	Server-side
Onde roda	Navegador do visitante	Servidor Veltrus
Bloqueio	Ad blockers afetam	Resiliente
Exemplo	gtag, fbq (se consentido)	CAPI, Measurement Protocol

GA4

Google Analytics 4 — eventos como `view_item`, `add_to_cart`, `begin_checkout`, `purchase`.

No storefront Saibai, eventos browser só disparam se `ga4_measurement_id` estiver preenchido e usuário consentiu analytics.

Meta CAPI

Conversions API — envia eventos do servidor para Meta, com deduplicação via `event_id` matching com o pixel browser.

Deduplicação

Mesmo evento (ex.: Purchase) não deve ser contado duas vezes. Por isso:

- IDs vazios no tema
- Purchase via **um** canal (Customer Events OU servidor, não ambos sem coordenação)

Risco do evento Purchase

Purchase é o evento mais crítico para ROAS. Duplicar infla conversões e destrói otimização de campanhas.

Por que IDs vazios no tema

Canais Shopify (Google & YouTube, Facebook & Instagram) já gerenciam pixels. Tema com IDs preenchidos = **segunda** tag = duplicação.

Testes antes de campanhas

- ☐ GTM Preview ou Meta Events Manager — PageView único
- ☐ Add to cart — um evento por ação
- ☐ Checkout teste — Purchase único
- ☐ POST `/apps/vlt-tracking/events` → **202**
- ☐ Consent negado → sem pixels marketing
- ☐ Consent aceito → pixels conforme configuração aprovada

9. Legal e Admin

Pages vs Policies

	<code>/pages/...</code>	<code>/policies/...</code>
Origem	Página criada no admin	Política legal Shopify
Uso Saibai	Conteúdo premium institucional	Políticas oficiais vinculadas ao checkout
Exemplo	<code>/pages/politica-de-entrega</code>	<code>/policies/shipping-policy</code>

Por que páginas duplicadas são ruins

Duas URLs com o mesmo conteúdo confundem o cliente, prejudicam SEO e criam risco de textos divergentes.

Por que mantivemos páginas premium existentes

Já tinham layout Saibai, SEO e links no footer — valor de UX superior ao gerador automático Shopify.

Boleto nos Termos

O storefront removeu referências a Boleto (método inativo no checkout Mercado Pago). O texto legal em **Termos de serviço** no Admin ainda pode mencionar boleto — item de backlog para aprovação do dono.

Por que pagamentos não foram alterados

Configuração de gateway (Mercado Pago, Pix, cartões) é decisão financeira do merchant. A Veltrus alinhou **apresentação** no site, não ativação/desativação de métodos.

10. Git, GitHub e PRs

Branch

Linha de desenvolvimento paralela. Ex.: `cursor/remove-boleto-footer-seal-70ba` para remover Boleto sem mexer na `main` diretamente.

Pull Request (PR)

Pedido de revisão para mesclar mudanças. Permite diff, comentários e histórico.

PR #2 — Boleto storefront

- **O quê:** Removeu selo e textos de Boleto do footer, PDP e carrinho
- **Status:** Publicado na live e **merged** na `main`
- **Por quê:** Boleto não está ativo no checkout — anunciar gerava frustração

PR #3 — Auditoria

- **Branch:** `audit/saibai-clean-theme-pass`
- **O quê:** `docs/AUDIT_SAIBAI_CLEAN_PASS.md` — inventário, achados, arquivos grandes
- **Status:** Documentação apenas — **não publicado** na live

PR #4 — Backlog

- **Branch:** `cursor/saibai-backlog-notes-70ba`
- **O quê:** `docs/BACKLOG_SAIBAI.md` — itens futuros
- **Status:** Documentação apenas — **não publicado** na live

Por que docs não vão para live

Documentação em `docs/` não faz parte do tema Shopify enviado à loja. PRs de doc não alteram comportamento do site — separar de PRs de tema evita risco acidental.

Publicação controlada na live

Tema live ID **#186124239166**. Publicar tema = impacto imediato em todos os visitantes. Sempre:

1. `shopify theme check`
2. Smoke test
3. Aprovação
4. `shopify theme push` ou `publish scoped`

Ler diffs com segurança

No GitHub PR, aba **Files changed**:

- Verde = adicionado
 - Vermelho = removido
 - Arquivos `docs/` = seguro para merge sem afetar loja
 - Arquivos `assets/`, `sections/`, `layout/` = exige QA
-

11. Checklist seguro para Everton

Antes de editar

- ☐ Entendi qual módulo estou alterando (seção 5)
- ☐ Estou em branch própria, não na `main`
- ☐ Li o arquivo e snippets relacionados
- ☐ Sei se é código Saibai ou nativo Ella
- ☐ Confirmo que não é área perigosa (seção 13)

Durante a edição

- ☐ Mudança mínima — uma coisa por vez
- ☐ Não criei arquivo CSS/JS novo sem aprovação (consolidar no existente)
- ☐ Não preenchi IDs de tracking no Theme Settings
- ☐ JSON válido (vírgulas, chaves)
- ☐ Comentário no código só se necessário

Depois de editar

- ☐ `shopify theme check` → 0 offenses
- ☐ Preview theme no navegador
- ☐ Testei desktop + mobile 375px
- ☐ Console sem erros novos
- ☐ Network sem 404 de assets

Antes de publicar

- ☐ PR criada com descrição clara
- ☐ Revisão de outra pessoa (Veltrus)
- ☐ Smoke test parcial do módulo afetado
- ☐ Backup: tema duplicado no admin Shopify

Depois de publicar

- ☐ Validar URL live (home, PDP, cart)
- ☐ Monitorar pedidos teste
- ☐ Verificar tracking (202 no proxy)
- ☐ Registrar o que mudou em `docs/` se relevante

12. Primeiros exercícios (seguros)

Exercício 1 — Editar um texto pequeno

1. Abra `sections/saibai-footer.liquid`
2. Localize `ft_n1_title` default: `'Receba novidades da safra'`
3. Em branch de teste, altere para `'Receba novidades da safra 2026'`
4. Preview → confirme no footer → reverta ou mantenha em PR de treino

Exercício 2 — Inspeccionar um snippet

1. Abra `snippets/saibai-logo.liquid`
2. Liste quais parâmetros aceita (`variant` , `layout` , `width`)
3. Busque onde é chamado: `grep -r "saibai-logo" sections/ snippets/`

Exercício 3 — Rastrear render de uma section

1. Home usa `templates/index.json` → `"type": "saibai-home-hero"`
2. Abra `sections/saibai-home-hero.liquid`

3. Siga `{% render %}` para snippets filhos
4. Desenhe o fluxo: template → section → snippets → assets

Exercício 4 — CSS no DevTools

1. Abra a loja preview
2. Inspeccione um card de produto
3. Na aba Styles, encontre quem define `aspect-ratio`
4. Anote arquivo origem (ex.: `saibai-shell-cards-media.css`)

Exercício 5 — Branch não-live

```
git checkout main
git pull
git checkout -b cursor/treino-everton-7c46
# faça uma edição trivial
git add -A && git commit -m "docs: treino Everton"
git push -u origin cursor/treino-everton-7c46
```

Exercício 6 — Theme check

```
cd /caminho/do/tema
shopify theme check
```

Meta: **0 offenses**.

Exercício 7 — PR só documentação

1. Edite apenas um arquivo em `docs/`
2. Abra PR com título `docs: ...`
3. Observe que nenhum arquivo de tema precisa ir para live

13. Áreas perigosas (não tocar sem revisão)

Área	Motivo
Configurações de pagamento	Impacto financeiro direto
Checkout / métodos de pagamento	Conversão e compliance PCI
Evento Purchase / tracking	ROAS e deduplicação
LGPD / banner consentimento	Risco legal
Frete e taxas	Custo para cliente
Textos legais	Risco jurídico
Loaders CSS (<code>saibai-shell-css</code> , <code>global-css</code>)	Quebra visual em massa
<code>cart-drawer.js</code> / <code>cart.js</code>	Carrinho para de funcionar
App embeds / <code>content_for_header</code>	Quebra apps e checkout
Publicar tema live	Impacto imediato em produção
<code>config/settings_data.json</code> tracking IDs	Duplica conversões

Remover arquivos nativos Ella	Regressões inesperadas
-------------------------------	------------------------

14. Glossário

Termo	Explicação para iniciante
Liquid	Linguagem de template da Shopify para misturar HTML com dados da loja
Snippet	Pedaço de código reutilizável incluído com <code>{% render %}</code>
Section	Bloco grande de página, configurável no Theme Editor
Schema	JSON que define campos editáveis de uma section no admin
JSON template	Arquivo que lista quais sections compõem uma página
Asset	Arquivo estático: CSS, JS, imagem
CSS cascade	Regras do navegador para decidir qual estilo prevalece
Override	Sobrescrever estilo/comportamento anterior
PR	Pull Request — proposta de mudança no Git para revisão
Branch	Linha de desenvolvimento separada no Git
Merge	Incorporar mudanças de uma branch em outra
Live theme	Tema publicado que visitantes veem em produção
Staging	Cópia do tema para testes antes de publicar
App Proxy	Rota <code>/apps/...</code> na loja que encaminha para servidor externo
Webhook	Notificação HTTP automática quando algo acontece na loja (ex.: pedido pago)
CAPI	Conversions API — tracking Meta pelo servidor
GA4	Google Analytics 4 — analytics e eventos de ecommerce
Consent Mode	Framework Google para respeitar consentimento antes de tracking
CDN cache	Cópia de arquivos em servidores globais para carregar mais rápido; pode atrasar ver mudanças

Referências no repositório

Documento	Uso
README.md	Visão geral e arquivos principais
docs/SMOKE_TEST_v1.2.4.md	Checklist QA antes de publicar
docs/DESPACHO_SAIBAI.md	Operação go-live e admin
docs/BACKLOG_SAIBAI.md	Próximas fases
docs/AUDIT_SAIBAI_CLEAN_PASS.md	Auditoria técnica
docs/CHECKOUT_BRANDING_GUIA.md	Branding checkout

